

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.М71-мм

# Разработка сервиса для работы с учебными практиками

*Каргин Глеб Павлович*

Отчёт по учебной практике  
в форме «Производственное задание»

Научный руководитель:  
старший преподаватель кафедры системного программирования, к.т.н. Ю. В. Литвинов

Санкт-Петербург  
2026

# Оглавление

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Введение</b>                                             | <b>3</b>  |
| <b>1. Постановка задачи</b>                                 | <b>5</b>  |
| <b>2. Обзор</b>                                             | <b>6</b>  |
| 2.1. Обзор применяемых технологий . . . . .                 | 6         |
| 2.2. Анализ предыдущей версии системы . . . . .             | 8         |
| <b>3. Проектирование</b>                                    | <b>11</b> |
| 3.1. Архитектура системы . . . . .                          | 11        |
| 3.2. Схема взаимодействия сервисов . . . . .                | 11        |
| <b>4. Реализация</b>                                        | <b>13</b> |
| 4.1. Сервис нотификации . . . . .                           | 13        |
| 4.2. Система логирования действий пользователей . . . . .   | 14        |
| 4.3. Механизмы гарантированной доставки сообщений . . . . . | 15        |
| 4.4. Сервис для анализа документов с использованием LLM .   | 16        |
| <b>5. Эксперимент</b>                                       | <b>17</b> |
| <b>6. Заключение</b>                                        | <b>18</b> |
| <b>Список литературы</b>                                    | <b>19</b> |

# Введение

В СПбГУ студенты математико-механического факультета, начиная со второго курса, проходят учебные практики и выполняют выпускные квалификационные работы (ВКР), направленные на углубление теоретических знаний и развитие профессиональных навыков. В этот процесс вовлечены несколько участников, каждый из которых играет важную роль. Студент отвечает за выполнение практики или ВКР, выбирает тему из предложенных направлений и согласовывает её с научным руководителем и, при необходимости, с консультантом. Научный руководитель — преподаватель, курирующий выполнение работы, а также помогающий в постановке задач. Консультант — специалист, обладающий узкой экспертизой в выбранной теме, который может быть как преподавателем СПбГУ, так и представителем внешней организации, и оказывает помощь по технической стороне работы. После завершения работы отчёт студента направляется на выборочное рецензирование, где рецензент оценивает соответствие требованиям, полноту и качество выполненного задания. За общую организацию и координацию процесса отвечает руководитель практики — сотрудник, отслеживающий соблюдение сроков и обеспечивающий взаимодействие между всеми участниками.

Однако процесс выбора тем, подачи отчётов и отслеживания выполнения практик остаётся трудоёмким и требует значительных усилий со стороны всех участников. В настоящее время на сайте кафедры системного программирования СПбГУ существует сервис, который призван автоматизировать часть этих процессов. Тем не менее, он имеет ряд существенных недостатков. Система сложна в поддержке: она разрабатывалась как дополнительная функциональность основного сайта кафедры, предназначенного в первую очередь для публикации статического контента, а не для организации и управления учебными процессами. По мере роста требований к процессу сопровождения практик и ВКР эта многоуровневая функциональность переросла возможности платформы, на которой была развёрнута изначально, что и создало

трудности как для разработчиков, так и для пользователей.

В связи с этим было принято решение создать отдельный сервис, независимый от основного сайта кафедры, который будет направлен исключительно на автоматизацию всех этапов учебной практики — от сбора и выбора тем до оценки результатов и размещения материалов успешных практик. Новый сервис предназначен для взаимодействия студентов, преподавателей и внешних экспертов на всех этапах процесса. Он включает сбор и проверку отчётов, а также контроль за выполнением задач.

Для реализации этой идеи разрабатывается специализированное веб-приложение с микросервисной архитектурой, что обеспечивает поддержку и возможность расширения функциональности системы для работы с учебными практиками СПбГУ. Разработка данного сервиса ведётся в рамках более крупного проекта, начатого автором в выпускной квалификационной работе [4]. Ввиду масштаба проекта в рамках данной учебной практики решается лишь часть задач по его развитию — они сформулированы в следующем разделе.

# 1. Постановка задачи

Целью работы является улучшение и доработка сервиса для работы с учебными практиками. Для её выполнения были поставлены следующие задачи.

1. Разработать сервис уведомлений.
2. Реализовать систему логирования действий пользователей.
3. Обеспечить надёжную доставку сообщений при асинхронном взаимодействии микросервисов.
4. Исправить имеющиеся недочеты системы.
5. Начать реализацию сервиса для анализа документов с использованием большой языковой модели (LLM).

## 2. Обзор

Поскольку задачи данной практики связаны с внедрением конкретных технических механизмов (гарантированная доставка сообщений, централизованное логирование, LLM-анализ документов), в этом разделе рассматриваются существующие решения именно в этих областях. Обзор систем-аналогов для управления учебными практиками был проведён ранее, в рамках выпускной квалификационной работы автора [4].

### 2.1. Обзор применяемых технологий

#### 2.1.1. Брокеры сообщений

Среди распространённых решений для организации асинхронного обмена сообщениями между компонентами распределённой системы можно выделить RabbitMQ [2], Apache Kafka [1] и ZeroMQ [3]. Apache Kafka ориентирована на потоковую обработку событий с долговременным хранением их истории и рассчитана на сценарии с высокой пропускной способностью и множеством независимых потребителей одного и того же потока данных. Для задач данного проекта, где сообщения представляют собой единичные события (регистрация, изменение статуса практики, комментарий к отчёту), подлежащие однократной обработке и не требующие повторного воспроизведения истории, такая модель избыточна и усложнила бы эксплуатацию системы. ZeroMQ, в отличие от RabbitMQ, представляет собой библиотеку для построения сетевых сообщений на уровне сокетов и не содержит таких функциональностей как буферизация сообщений при недоступности потребителя, подтверждения обработки, повторные попытки. RabbitMQ, в свою очередь, предоставляет данные механизмы в готовом виде, обладает клиентской библиотекой для .NET и поддерживает гибкую маршрутизацию сообщений через различные типы обменников. На основании изложенного в качестве брокера сообщений был выбран RabbitMQ.

### 2.1.2. Решения для хранения и анализа логов

Стандартным подходом к организации централизованного логирования в индустрии является использование специализированного стека, например Elasticsearch-Logstash-Kibana (ELK) или его облегчённых аналогов (Grafana Loki, Graylog). Такие решения предоставляют полнотекстовый поиск по логам, построение дашбордов и уведомления в реальном времени, однако требуют развёртывания и сопровождения нескольких дополнительных компонентов инфраструктуры (индексирующего хранилища, конвейера сбора логов, средства визуализации). Для задач данной практики, сохранения и последующего выборочного просмотра администратором записей о действиях пользователей, такой набор возможностей избыточен, а сопровождение полноценного стека несоразмерно затратам на его развёртывание. Поэтому было принято решение не разворачивать готовые решения для логирования, а реализовать собственный сервис аудита (Audit Service), использующий существующую в проекте инфраструктуру обмена сообщениями (RabbitMQ) для сбора событий и документоориентированную СУБД для их хранения. Такой подход позволяет ограничиться одним дополнительным компонентом вместо нескольких, при этом сохраняя гибкость схемы хранения, необходимую из-за разной структуры деталей у разных типов событий.

### 2.1.3. Библиотеки для отправки email-уведомлений

Для интеграции с SMTP-сервером в экосистеме .NET можно выделить несколько распространённых вариантов: встроенный класс `System.Net.Mail.SmtpClient`, официально признанный устаревшим и не рекомендуемый Microsoft к использованию в новых проектах; библиотеку `MailKit`, активно поддерживаемую и предоставляющую асинхронный API, полноценную поддержку MIME-сообщений и HTML-форматирования; а также сторонние облачные сервисы рассылки (например, `SendGrid`), предполагающие внешнюю зависимость от платного API и избыточные для университетского сервиса с невысокой нагрузкой. Ввиду прекращения поддержки `SmtpClient` и нежелательности

внешней зависимости от коммерческого сервиса для отправки email-уведомлений выбрана библиотека MailKit.

## 2.2. Анализ предыдущей версии системы

На начальном этапе эксплуатации версии MVP (Minimum Viable Product)<sup>1</sup> был выявлен ряд архитектурных недочетов и технических проблем, требующих устранения для обеспечения корректной работы системы в промышленной эксплуатации.

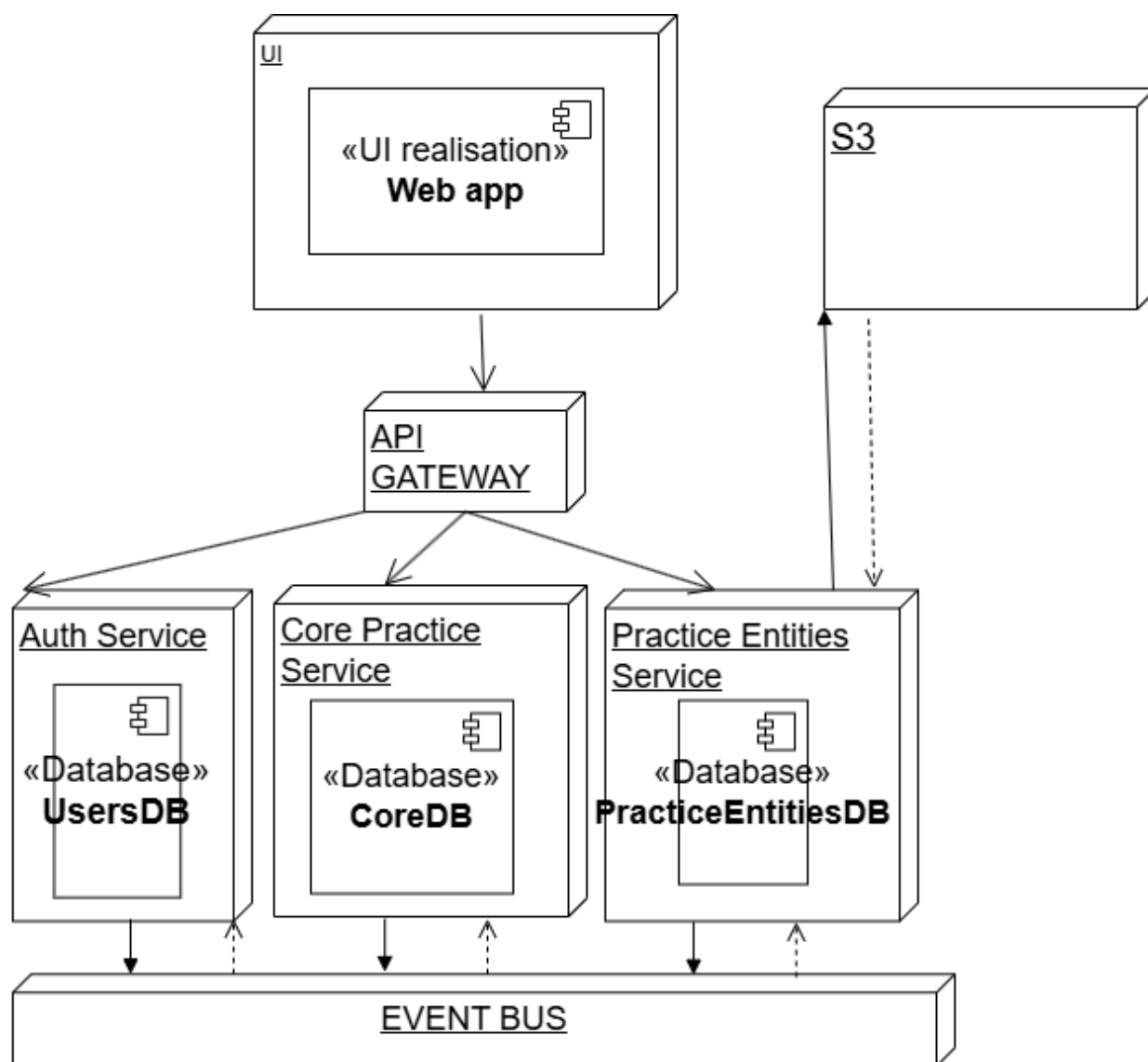


Рис. 1: Диаграмма предыдущей версии

<sup>1</sup>MVP. URL: [https://ru.wikipedia.org/wiki/Минимально\\_жизнеспособный\\_продукт](https://ru.wikipedia.org/wiki/Минимально_жизнеспособный_продукт) (дата обращения: 06.07.2026)



Анализ существующей реализации показал, что, несмотря на работоспособность, система нуждается в ряде доработок архитектурного характера.

### **2.2.1. Асинхронное взаимодействие микросервисов**

Несмотря на то что брокер сообщений RabbitMQ был добавлен в архитектуру системы ранее, вопросы гарантированной доставки и надежности асинхронного взаимодействия остались нерешёнными. Это приводило к нарушению согласованности данных при недоступности сервисов или временных сбоях в сети. Отсутствие подтверждений доставки и механизмов повторных попыток обработки сообщений создавало риск потери важных событий. Для решения данной проблемы требовалась корректная настройка брокера сообщений, обеспечивающего гарантированную доставку событий между микросервисами.

### **2.2.2. Сервис нотификации**

В текущей версии отсутствовал централизованный сервис, отвечающий за отправку уведомлений пользователям системы. Такие функции, как подтверждение регистрации, восстановление пароля или оповещение об изменении статуса практики, не были реализованы в полном объеме. Требуется создание отдельного микросервиса, который бы обеспечивал отправку email-уведомлений и интеграцию с брокером сообщений для асинхронной обработки запросов.

### **2.2.3. Логирование**

Отсутствие единой системы сбора и хранения информации о действиях пользователей и системных событиях затрудняет анализ инцидентов и аудит безопасности. Для обеспечения возможности отслеживания критических операций (вход в систему, изменение ролей, регистрация новых пользователей) необходимо внедрение централизованного сервиса логирования с выделенным хранилищем.

#### **2.2.4. Автоматизация проверки документов**

Перспективным направлением развития является внедрение интеллектуальных механизмов анализа отчетов, презентаций и отзывов. Автоматизация этого процесса с использованием технологий искусственного интеллекта позволит снизить нагрузку на преподавателей и повысить качество обратной связи для студентов. Для реализации такой функциональности требуется создание отдельного сервиса, интегрирующегося с существующей инфраструктурой и использующего большие языковые модели для анализа содержимого документов.

Таким образом, анализ текущего состояния системы позволил определить ключевые направления ее развития, которые будут рассмотрены в следующих разделах данной работы.

## 3. Проектирование

### 3.1. Архитектура системы

На основе анализа, проведенного в предыдущем разделе, в архитектуру системы были внесены изменения, направленные на устранение выявленных недостатков. В состав системы добавлены следующие компоненты:

- **Сервис нотификации (Notification Service)** — обеспечивает централизованную отправку email-уведомлений, включая подтверждение регистрации, восстановление пароля и оповещения об изменениях в практиках.
- **Сервис логирования (Audit Service)** — осуществляет сбор и хранение информации о действиях пользователей и системных событиях. Данный компонент позволяет отслеживать критически важные операции (вход в систему, изменение ролей, регистрация новых пользователей) и фиксировать возникающие ошибки, что повышает прозрачность системы и упрощает анализ инцидентов.
- **Сервис анализа документов (LLM Service)** — обеспечивает автоматическую проверку загружаемых документов (отчетов, презентаций, отзывов) с использованием больших языковых моделей.

Для организации надежного асинхронного взаимодействия между компонентами используется брокер сообщений RabbitMQ, обеспечивающий гарантированную доставку событий.

### 3.2. Схема взаимодействия сервисов

Взаимодействие между компонентами системы организовано следующим образом. Клиентские запросы поступают на API Gateway, который на основе маршрутизации перенаправляет их к соответствующим сервисам. Аутентификация и авторизация выполняются в Auth Service,

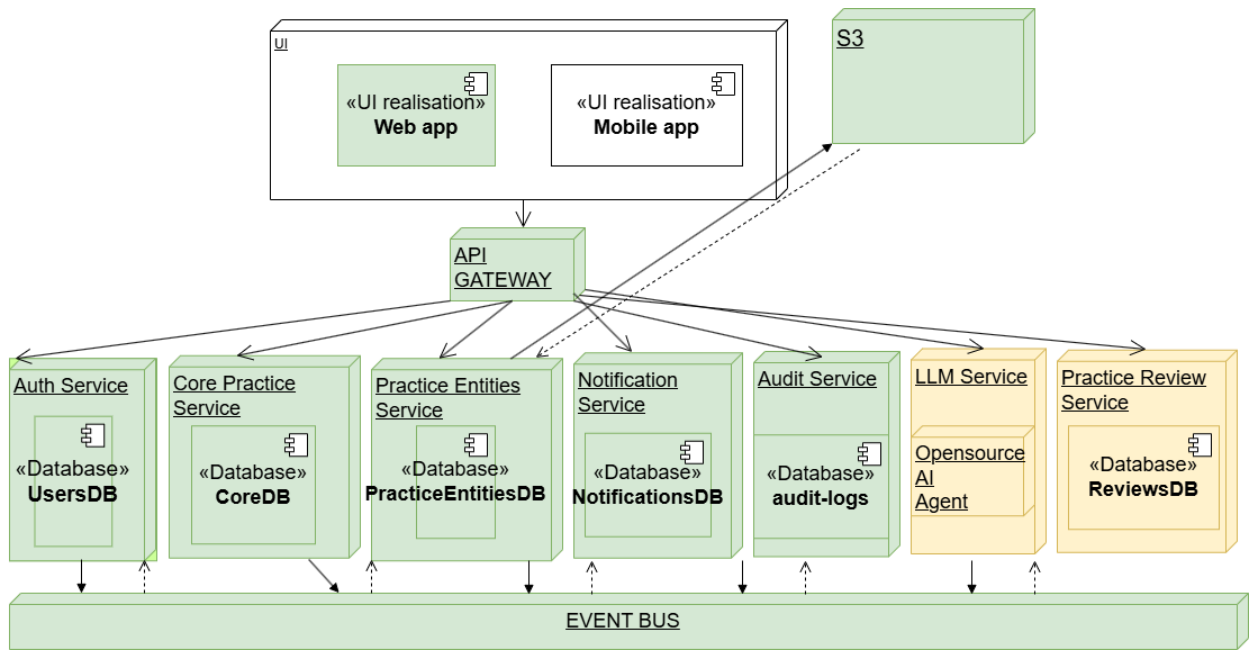


Рис. 2: Обновленная диаграмма системы. Зеленым цветом выделены реализованные компоненты

который выдает JWT-токены, используемые для защиты конечных точек.

При изменении состояния сущностей (создание пользователя, назначение роли, изменение практики) сервисы публикуют события в RabbitMQ. Это позволяет другим компонентам системы реагировать на изменения асинхронно. Например, при изменении роли пользователя публикуется событие, которое обрабатывается Core Service для синхронизации профиля, а также Notification Service для отправки уведомления.

Сервис аудита подписан на все значимые события и сохраняет их в отдельном хранилище для последующего анализа. Такой подход позволяет централизованно логировать действия пользователей без внесения изменений в бизнес-логику сервисов.

## 4. Реализация

В рамках данной работы были реализованы следующие компоненты системы.

### 4.1. Сервис нотификации

Одной из ключевых задач являлась разработка микросервиса для централизованной отправки email-уведомлений. Данный компонент отвечает за оповещение пользователей о различных событиях в системе: подтверждение регистрации, восстановление пароля, изменение статуса практик и другие уведомления.

Сервис нотификации построен как отдельный микросервис, взаимодействующий с остальными компонентами системы через брокер сообщений RabbitMQ. Такой подход позволяет обрабатывать запросы на отправку писем асинхронно, не блокируя выполнение основных бизнес-операций.

Архитектура сервиса включает потребителей сообщений (Consumers) — каждый тип уведомления обрабатывается отдельным потребителем, подписанным на соответствующую очередь в RabbitMQ. Это позволяет изолировать обработку различных сценариев и упрощает масштабирование. Кроме того, в состав сервиса входит Email Service, отвечающий за формирование и отправку электронных писем через SMTP-сервер с поддержкой HTML-форматирования для создания структурированных уведомлений.

В рамках реализации сервиса были созданы потребители для следующих типов событий:

- `EmailConfirmationConsumer` — отправка ссылки для подтверждения электронной почты при регистрации;
- `PasswordResetRequestedConsumer` — отправка ссылки для восстановления пароля;
- `PracticeUpdatedConsumer` — уведомление об изменении статуса

практики;

- `FeedbackSubmittedConsumer` — оповещение о новом отзыве;
- `MessageSentConsumer` — уведомление о новом сообщении;
- `TextWorkSubmittedConsumer` — оповещение о загрузке текстовой работы;
- `TextWorkCommentAddedConsumer` — уведомление о новом комментарии к текстовой работе;
- `PresentationSubmittedConsumer` — уведомление о загрузке презентации;
- `PresentationCommentAddedConsumer` — уведомление о новом комментарии к презентации;
- `ReportSubmittedConsumer` — оповещение о загрузке отчета;
- `ReportCommentAddedConsumer` — уведомление о новом комментарии к отчету;
- `ThemeArchivedConsumer` — уведомление об архивации темы;
- `GoalsAndTasksUpdatedConsumer` — оповещение об обновлении целей и задач практики.

Каждый потребитель обрабатывает входящее событие и формирует соответствующее письмо, которое затем отправляется через SMTP-клиент. Для взаимодействия с SMTP-сервером используется библиотека MailKit.

## 4.2. Система логирования действий пользователей

Для обеспечения мониторинга и аудита действий пользователей был реализован отдельный микросервис Audit Service, выполняющий централизованный сбор и хранение информации о событиях, происходящих в системе.

Сбор событий организован таким образом, что каждый значимый компонент системы публикует события в RabbitMQ при выполнении критических операций: вход в систему, изменение ролей, регистрация новых пользователей, создание и редактирование практик. Обработка событий осуществляется потребителем `AuditConsumer`, который принимает события из очереди и сохраняет их в базе данных. Для каждого события фиксируется идентификатор пользователя, его email, роли, тип выполненного действия, идентификатор сущности, дополнительные параметры, статус выполнения (успех/ошибка) и время события. Хранение аудиторских записей реализовано на базе MongoDB. В отличие от стека Elasticsearch-Logstash-Kibana, часто применяемого как стандартное решение для хранения и анализа логов, MongoDB выбрана исходя из того, что для задач практики не требовался полнотекстовый поиск и визуализация логов в реальном времени, а требовалась гибкая схема документа для хранения разноструктурных деталей событий (тип действия, идентификатор сущности и специфичный для каждого действия набор параметров) без развёртывания дополнительной многокомпонентной инфраструктуры.

Для обеспечения быстрого доступа к данным в коллекции `audit_logs` созданы индексы по полям `Action` (тип действия) и `UserEmail` (пользователь). Это позволяет выполнять фильтрацию и поиск записей с минимальной задержкой.

Сервис аудита предоставляет конечные точки для получения информации, доступ к которым ограничен политикой `AdminOnly`, что обеспечивает просмотр логов только администраторами системы.

### 4.3. Механизмы гарантированной доставки сообщений

Для обеспечения надежности асинхронного взаимодействия между микросервисами в RabbitMQ были внедрены механизмы, повышающие отказоустойчивость системы. Для каждого потребителя настроены политики повторных попыток (Retry Policies) — например, при сбое

SMTP-сервера в сервисе нотификации сообщение повторно обрабатывается через заданные интервалы (1, 2, 5 секунд) до успешного выполнения или исчерпания лимита попыток. Очереди разделены по типам событий, что позволяет гибко настраивать приоритеты обработки: критически важные уведомления (сброс пароля) обрабатываются с более высоким приоритетом и меньшим количеством параллельных потребителей, чем менее значимые события (фидбеки). Кроме того, настроены параметры Heartbeat и таймауты подключения, обеспечивающие автоматическое переподключение при потере связи с брокером.

#### **4.4. Сервис для анализа документов с использованием LLM**

Для автоматизации проверки документов (отчетов, презентаций, отзывов) была начата разработка отдельного микросервиса, использующего большие языковые модели (LLM). Данный компонент предназначен для предоставления пользователям автоматической обратной связи по качеству оформления и содержанию загружаемых документов.

В рамках начального этапа реализации были созданы базовые агенты для анализа отчетов и проверки орфографии. Дальнейшее развитие предполагает расширение набора агентов, внедрение в процесс загрузки документов и улучшение качества анализа.



## 5. Эксперимент

Работоспособность механизма повторных попыток была проверена вручную: в конфигурации сервиса нотификации временно указывались заведомо недействительные учётные данные для SMTP-сервера, после чего инициировалось событие, требующее отправки уведомления. Было зафиксировано, что сообщение не терялось, а оставалось в очереди и повторно обрабатывалось потребителем с заданными интервалами (1, 2, 5 секунд). После возврата корректных учётных данных письмо было успешно доставлено при следующей попытке обработки.

Кроме того, была проверена корректность работы сервиса нотификации: для нескольких типов событий (подтверждение email, восстановление пароля, изменение статуса практики) вручную инициировалась публикация соответствующего события, после чего проверялось, что сервис формировал и отправлял письмо с ожидаемым содержанием.

Также была проверена корректность работы сервиса аудита: при выполнении критических операций (вход в систему, регистрация, изменение роли пользователя) было зафиксировано появление соответствующей записи в коллекции `audit_logs` с корректно заполненными полями (тип действия, email пользователя, время события).

Дополнительно была проверена устойчивость сервиса нотификации к временной недоступности брокера сообщений: после остановки контейнера RabbitMQ сервис фиксировал потерю соединения и предпринимал попытки переподключения; после повторного запуска RabbitMQ соединение восстанавливалось автоматически, без ручного перезапуска сервиса.

## 6. Заключение

В ходе выполнения данной работы были получены следующие результаты.

- Обновлено архитектура системы с добавлением новых компонентов.
- Разработан сервис нотификации для централизованной отправки email-уведомлений.
- Реализована система логирования действий пользователей.
- Добавлена надёжная доставка сообщений при асинхронном взаимодействии микросервисов.
- Начата реализация сервиса для анализа документов с использованием LLM.

Код доступен в репозитории на GitHub<sup>2</sup>.

---

<sup>2</sup>PracticesService. URL: <https://github.com/spbu-se/PracticesService/tree/develop> (дата обращения: 2026-07-06). [Имя аккаунта: Belgrak]

## Список литературы

- [1] Apache Kafka. — URL: <https://kafka.apache.org/> (дата обращения: 2026-07-06).
- [2] RabbitMQ. — URL: <https://www.rabbitmq.com/> (дата обращения: 2026-07-06).
- [3] ZeroMQ. — URL: <https://zeromq.org/> (дата обращения: 2026-07-06).
- [4] Каргин Г. П. Разработка сервиса для работы с учебными практиками. — URL: [https://se.math.spbu.ru/thesis/texts/Kargin\\_Gleb\\_Pavlovich\\_Bachelor\\_Thesis\\_2025\\_text.pdf](https://se.math.spbu.ru/thesis/texts/Kargin_Gleb_Pavlovich_Bachelor_Thesis_2025_text.pdf) (дата обращения: 6 июля 2026 г.).